Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2
По дисциплине «Проектирование программного обеспечения интеллектуальных систем»
За 1 семестр

Тема: «Перегрузка функций. Разработка и реализация пользовательских классов»

Выполнил:
Студент ФЭИС
Группы ИИ-29
Годованный Ю.М.
Проверил:
Монтик Н.С.

Брест 2026

Цель работы: Изучить механизмы перегрузки функций; приобрести практические навыки программирования пользовательских классов.

Ход работы:

Задание: На основе класса `Student`, реализованного в лабораторной работе №1, продемонстрировать следующие виды перегрузки функций: с разным количеством параметров; с разными типами параметров; с сочетанием различий по числу и типу параметров; с использованием `const`-квалификаторов; а также показать невозможность перегрузки только по возвращаемому типу. Перегрузка операторов в данной работе не используется.

Описание реализованных перегрузок:

Перегрузки добавлены в класс `Student` (предметная область «Студент»; поля: `course` - курс, `group` - группа, `grades[SIZE]` - статический массив оценок). Конструкторы, метод `TO_SHOW` и операторы из лабораторной работы №1 сохранены. В функции `main()` перегрузки вызываются для объектов `s`, `s2` и константного объекта `cs`.

1. Разное количество параметров.

- `void TO_SET_Grade(int i, int value)` - устанавливает оценку с индексом `i`;
- `void TO_SET_Grade(int value)` - устанавливает одно значение для всех оценок.

Сигнатуры различаются числом параметров. Компилятор выбирает функцию по числу аргументов в вызове: `TO_SET_Grade(0, 6)` - первая версия, `TO_SET_Grade(5)` - вторая.

2. Разные типы параметров.

- `void TO_SET_Grade(int i, int value)` - целая оценка;
- `void TO_SET_Grade(int i, double value)` - вещественная оценка, которая округляется до ближайшего целого.

Число параметров одинаково, различается тип второго. Компилятор выбирает версию с точным совпадением типов аргументов:

`TO_SET_Grade(1, 7)` - версия с `int`, `TO_SET_Grade(2, 7.6)` - версия с `double` (оценка 8). Без перегрузки `double` значение 7.6 было бы молча усечено до 7.

3. Различие и по числу, и по типу параметров.

- `void TO_SET_Grade(double value)` - записывает округлённое значение во все оценки.

Эта сигнатура отличается от `(int, int)` и числом параметров, и типом первого параметра, а от `(int)` - типом параметра. В вызове `TO_SET_Grade(9.4)` подходят версии с одним параметром: `(int)` и `(double)`; точное совпадение типов у `(double)`, поэтому выбирается она (результат: все оценки равны 9).

4. Перегрузка с использованием const.

4.1. `const`-квалификатор метода (относится к неявному параметру `this`):

- `int TO_GET_Grade(int i) const` - для `const`-объектов; гарантирует, что объект не изменяется, возвращает копию значения;
- `int& TO_GET_Grade(int i)` - для изменяемых объектов; возвращает ссылку на оценку, поэтому её можно изменить: `s2.TO_GET_Grade(1) = 10;`.

Для `const`-объекта `cs` компилятор может вызвать только `const`-версию. Для обычного объекта `s2` из двух подходящих функций выбирается не-`const` версия, так как она точнее соответствует типу `this`. Благодаря перегрузке `const`-объекты можно читать, не открывая возможности их изменения.

4.2. `const`-квалификатор параметра-ссылки:

- `int TO_COMPARE(Student& other) const` - аргумент: обычный объект;
- `int TO_COMPARE(const Student& other) const` - аргумент: `const`-объект или временный объект.

Метод сравнивает суммы оценок и возвращает 1, -1 или 0. Типы `Student&` и `const Student&` - разные, поэтому это две разные функции. Обычный объект `s` привязывается к `Student&`, а `const`-объект `cs` и временный объект `Student(1, 1, g)` - только к `const Student&`. Квалификатор `const` при передаче по значению (`f(int x)` и `f(const int x)`) перегрузкой не является: сигнатуры совпадают.

5. Перегрузка только по возвращаемому типу.

Объявление двух методов, различающихся лишь типом результата, не компилируется:

```
class Student {
private:
    int course;
public:
    Student() { course = 1; }
    int TO_GET_Course() const { return course; }
    double TO_GET_Course() const { return course; }
};
```

Выражение `s.TO_GET_Course()` одинаково для обеих версий, а тип результата при выборе функции не учитывается (значение вызова может вообще не использоваться), поэтому выбор был бы неоднозначен. Компилятор сообщает об ошибке уже при объявлении второй функции:

```
error: 'double Student::TO_GET_Course() const' cannot be
       overloaded with 'int Student::TO_GET_Course() const'
note: previous declaration 'int Student::TO_GET_Course() const'
```

Как компилятор выбирает функцию. Сначала составляется список функций с подходящим именем; затем из него остаются те, которым подходят число и типы аргументов (с учётом допустимых преобразований и `const`); из оставшихся выбирается функция с наилучшим соответствием: точное совпадение типов лучше преобразования. Если единственной лучшей функции нет, выдаётся ошибка неоднозначности.

Код программы:

```
#include <iostream>
#include <cstdlib>
#include <clocale>
using namespace std;
```

```
const int SIZE = 5;

class Student {
private:
    int course;
    int group;
    int grades[SIZE];

    static int Round(double value) {
        return (int)(value + 0.5);
    }

public:
    Student() {
        course = 1;
        group = 0;
        for (int i = 0; i < SIZE; i++) {
            grades[i] = 0;
        }
    }

    Student(int c, int g, const int arr[]) {
        course = c;
        group = g;
        for (int i = 0; i < SIZE; i++) {
            grades[i] = arr[i];
        }
    }

    Student(const Student& other) {
        course = other.course;
        group = other.group;
        for (int i = 0; i < SIZE; i++) {
            grades[i] = other.grades[i];
        }
    }

    void TO_SHOW() const {
        cout << "Курс: " << course;
        cout << " | Группа: " << group;
        cout << " | Оценки: ";
        for (int i = 0; i < SIZE; i++) {
            cout << grades[i] << " ";
        }
        cout << endl;
    }

    int TO_GET_Course() const { return course; }
    int TO_GET_Group() const { return group; }
    void TO_SET_Course(int c) { course = c; }
    void TO_SET_Group(int g) { group = g; }
```

```
int TO_GET_Sum() const {
    int sum = 0;
    for (int i = 0; i < SIZE; i++) {
        sum += grades[i];
    }
    return sum;
}

void TO_SET_Grade(int i, int value) {
    if (i >= 0 && i < SIZE) {
        grades[i] = value;
    }
}

void TO_SET_Grade(int value) {
    for (int i = 0; i < SIZE; i++) {
        grades[i] = value;
    }
}

void TO_SET_Grade(int i, double value) {
    if (i >= 0 && i < SIZE) {
        grades[i] = Round(value);
    }
}

void TO_SET_Grade(double value) {
    int rounded = Round(value);
    for (int i = 0; i < SIZE; i++) {
        grades[i] = rounded;
    }
}

int TO_GET_Grade(int i) const {
    if (i < 0 || i >= SIZE) return -1;
    return grades[i];
}

int& TO_GET_Grade(int i) {
    static int stub = 0;
    if (i < 0 || i >= SIZE) {
        stub = 0;
        return stub;
    }
    return grades[i];
}

int TO_COMPARE(Student& other) const {
    cout << "[TO_COMPARE(Student&)]";
    if (TO_GET_Sum() > other.TO_GET_Sum()) return 1;
    if (TO_GET_Sum() < other.TO_GET_Sum()) return -1;
```

```
        return 0;
    }

    int TO_COMPARE(const Student& other) const {
        cout << "[TO_COMPARE(const Student&)]";
        if (TO_GET_Sum() > other.TO_GET_Sum()) return 1;
        if (TO_GET_Sum() < other.TO_GET_Sum()) return -1;
        return 0;
    }

};

int main() {
    system("chcp 65001");
    setlocale(LC_ALL, "ru_RU.UTF-8");

    int g[SIZE] = {8, 9, 7, 10, 9};
    Student s(2, 29, g);
    cout << "=== Исходный объект ===" << endl;
    s.TO_SHOW();

    cout << endl;
    cout << "=== 1. Разное количество параметров ===" << endl;

    s.TO_SET_Grade(0, 6);
    cout << "TO_SET_Grade(0, 6):   ";
    s.TO_SHOW();

    s.TO_SET_Grade(5);
    cout << "TO_SET_Grade(5):      ";
    s.TO_SHOW();

    cout << endl << "=== 2. Разные типы параметров ===" << endl;

    s.TO_SET_Grade(1, 7);
    cout << "TO_SET_Grade(1, 7):   ";
    s.TO_SHOW();

    s.TO_SET_Grade(2, 7.6);
    cout << "TO_SET_Grade(2, 7.6): ";
    s.TO_SHOW();

    cout << endl << "=== 3. Число и тип параметров ===" << endl;

    s.TO_SET_Grade(9.4);
    cout << "TO_SET_Grade(9.4):    ";
    s.TO_SHOW();

    cout << endl << "=== 4. Перегрузка с const ===" << endl;

    int h[SIZE] = {6, 7, 8, 9, 10};
```

```
    const Student cs(3, 15, h);
    Student s2(2, 29, g);

    cout << "const-объект, оценка №2: ";
    cout << cs.TO_GET_Grade(1) << endl;

    s2.TO_GET_Grade(1) = 10;
    cout << "s2 после TO_GET_Grade(1) = 10: ";
    s2.TO_SHOW();

    int r;

    r = s2.TO_COMPARE(s);
    cout << " результат: " << r << endl;

    r = s2.TO_COMPARE(cs);
    cout << " результат: " << r << endl;

    r = s2.TO_COMPARE(Student(1, 1, g));
    cout << " результат: " << r << endl;

    return 0;
}
```

Результат:

```
Active code page: 65001
=== Создание объектов ===
Объект 1 (по умолчанию): Курс: 1 | Группа: 0 | Оценки: 0 0 0 0 0
Объект 2 (с параметрами): Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9
Объект 3 (копия объекта 2): Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9

=== Методы TO_SET ===
Объект 1 после TO_SET: Курс: 1 | Группа: 15 | Оценки: 6 7 5 8 6
Объект 3 после TO_SET_Grade(4, 5): Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 5
Объект 2 (не изменился): Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9

=== Методы TO_GET ===
Курс объекта 2: 2
Группа объекта 2: 29
Оценка №4 объекта 2: 10

=== Оператор = ===
Объект 1 после s1 = s2: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9

=== Бинарный оператор + ===
Результат s2 + s3: Курс: 2 | Группа: 29 | Оценки: 16 18 14 20 14

=== Унарный оператор ++ ===
Объект 3 до ++: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 5
Объект 3 после ++: Курс: 3 | Группа: 29 | Оценки: 8 9 7 10 5
Press any key to continue . . .
```

Вывод: Изучили механизмы перегрузки функций и применили их в классе `Student`: реализовали перегрузки с разным количеством параметров, с разными типами параметров, с сочетанием этих различий и с использованием `const`-квалификаторов метода и параметра-ссылки. Показали, что перегрузка только по возвращаемому типу невозможна: компилятор не учитывает тип результата при выборе функции и выдаёт ошибку.